

BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION COMPUTER SCIENCE IN
ENGLISH

DIPLOMA THESIS

Fractals and GPU Computing

Supervisor
Assoc. Prof. Conf Rareș Florin Boian Ph. D.

Author
Cernăuțan Teofan Ioan

2026

**UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICA IN LIMBA
ENGLEZA**

LUCRARE DE LICENȚĂ

Fractali și Procesare pe GPU

**Conducător științific
Assoc. Prof. Conf Rareș Florin Boian Ph. D.**

*Absolvent
Cernăuțan Teofan Ioan*

2026

ABSTRACT

This thesis explores the general fractal and the usage of the GPU to compute using different parallel processing techniques to visualize and render using complex equations. The work begins with an overview of fractals, their history, mathematical foundation, types, and use cases. It then continues into the GPU computing side, from information about the CUDA architecture and performance optimization strategies. As a practical example, the thesis presents the implementation of a Julia fractal rendering program using CUDA as architecture. Although the work builds upon existing methods and techniques, it emphasizes experimentation and exploration of ways to improve performance through optimization.

Contents

1	Introduction	1
1.1	Overview of the Main Idea	1
1.2	Relevance and Motivation for the Topic	1
1.2.1	Why It's Important	2
1.2.2	What Benefits It Brings	2
1.3	Existing Work and Background Context	2
1.4	Personal Contribution	2
1.5	Structure of the Thesis	3
1.6	Use of Generative AI Tools	3
2	Fractals — Theory, Types, and Applications	4
2.1	A Brief History of Fractals	4
2.2	Mathematical Foundations	6
2.2.1	Self-Similarity and Recursion	6
2.2.2	Fractal Dimension	7
2.3	Types of Fractals	7
2.3.1	Geometric Fractals	8
2.3.2	Complex Fractals	8
2.3.3	Random or Natural Fractals	9
2.4	Applications of Fractals	11
2.4.1	Computer Graphics and Visualization	11
2.4.2	Natural Sciences	12
2.4.3	Signal Processing and Data Compression	13
2.4.4	Fractals in Art and Popular Culture	13
3	GPU Computing and Fractals: First Implementation	15
3.1	Introduction to GPU Computing	15
3.1.1	Why Use the GPU?	15
3.1.2	Advantages Over CPU for Fractal Generation	16
3.2	CUDA: Overview and Setup	17
3.2.1	Experimental Setup	17

3.2.2	Installation and Environment	17
3.2.3	CUDA Programming Model	18
3.3	The First Fractal: Julia Set	18
3.3.1	Mathematical Definition	18
3.3.2	Why the Julia Set Was Chosen	19
3.4	Code Architecture	20
3.4.1	Main Components	20
3.4.2	Memory Usage and Data Flow	23
3.5	Initial Results	25
3.5.1	Output Quality	25
3.5.2	Performance Observations	27
3.6	Concluding Remarks	29
4	GPU Optimization Strategies for Fractal Rendering	30
4.1	Introduction and Optimization Motivation	30
4.1.1	Overview of Implementation	30
4.1.2	Why It Still Matters	31
4.2	Step-by-Step GPU Optimizations	31
4.2.1	Unified Memory Access	31
4.2.2	Switch to Pitched Memory	32
4.2.3	Thread Block Configuration Improvement	35
4.3	Comparative Analysis and Results	36
4.3.1	Execution Time Benchmarks	36
4.3.2	Interpretation of Results	40
4.4	Discussion and Future Directions	40
4.4.1	Overview and Achievements	40
4.4.2	Optimization Possibilities and Future Work	41
5	Conclusions	42
	Bibliography	43

Chapter 1

Introduction

1.1 Overview of the Main Idea

Fractals are intricate mathematical structures that exhibit self-similarity and infinite detail, often described through recursive or iterative processes. These patterns are not only aesthetically fascinating, but also carry deep mathematical significance and practical applications. With the rise of high-performance computing, especially in the form of Graphics Processing Units (GPUs), the possibility of rendering complex fractals in real time has become more accessible. This thesis focuses on the intersection between fractal mathematics and GPU computing. The purpose of this paper is to present the fundamental concepts behind fractals and to explore how parallel processing techniques using CUDA-enabled GPUs can be leveraged to render these structures efficiently. By combining mathematical theory with computational optimization, this work highlights the benefits of GPU-accelerated fractal generation and the challenges involved in implementing such systems.

1.2 Relevance and Motivation for the Topic

The motivation behind this thesis arises from both the beautiful visuals and computational complexity of fractals, as well as the increasing importance of GPU computing in solving intensive mathematical problems in various cases, AI being the rising star. As the demand for real-time simulations and graphical applications continues to grow, efficient algorithms for rendering complex visuals have become an important area of research. Fractals provide a perfect test case for GPU-based computing due to their recursive nature, high computational cost, and potential for parallel execution.

1.2.1 Why It's Important

Studying the connection between fractals and GPU computing provides a useful lens for understanding the power and limitations of modern hardware acceleration. It encourages the application of abstract mathematical theory in a very practical and visual way. Moreover, by implementing fractal rendering on the GPU, developers can achieve significant performance improvements over CPU-based approaches, which is relevant not just for graphics, but also for fields like signal processing, procedural generation, and simulations.

1.2.2 What Benefits It Brings

This topic offers educational value by reinforcing both mathematical and programming concepts while demonstrating how parallelism can solve real-world problems. It also shows how tools such as CUDA allow low-level control over hardware to maximize performance. Understanding how to optimize fractal rendering on the GPU can also serve as a stepping stone to other performance-critical applications beyond graphics.

1.3 Existing Work and Background Context

Fractal geometry has been extensively studied since the work of mathematicians like Benoît Mandelbrot. Many types of fractals, such as the Mandelbrot set, Julia sets, and L systems, have been visualized through various algorithms. In terms of GPU computing, CUDA has enabled significant advances in accelerating such computations, with numerous academic and personal projects focusing on real-time rendering and interactive fractal exploration. However, most of the existing work is highly theoretical or very specialized; this thesis aims to provide a more balanced and accessible introduction to both sides.

1.4 Personal Contribution

This work does not introduce new mathematical theories or hardware innovations. Instead, its primary contribution is the integration of existing knowledge into a coherent and practical example. By developing a fractal rendering application using CUDA, the thesis demonstrates effective use of GPU parallelism for fractal visualization. Additional contributions include performance improvements through optimization techniques, and thorough documentation aimed at serving as a valuable reference for students and beginners in GPU computing and fractal rendering.

1.5 Structure of the Thesis

The thesis begins with an introduction outlining the motivation and personal contributions. Chapter 2 covers fractals in detail, presenting their mathematical foundations, historical development, classification, and practical applications. Chapter 3 focuses on GPU computing fundamentals and presents the initial implementation of a fractal rendering program using CUDA. Chapter 4 explores optimization strategies to improve fractal rendering performance on GPUs, including benchmarking and color mapping experiments. Finally, Chapter 5 concludes the thesis by summarizing key findings, discussing challenges faced, and suggesting directions for future work.

1.6 Use of Generative AI Tools

During the preparation of this work, the author used ChatGPT in order to assist with technical writing, LaTeX formatting, and text clarity. After using this tool/service, the author reviewed and edited the content as needed and assumes full responsibility for the content of the thesis.

Chapter 2

Fractals — Theory, Types, and Applications

2.1 A Brief History of Fractals

Although the term fractal was only coined in the 1970s by Benoît Mandelbrot, the underlying ideas behind fractals, such as proportion, symmetry, and self-similarity, can be traced back to antiquity. Ancient Greek thinkers, including Pythagoras, Euclid, and Plato, made foundational contributions to mathematics and geometry that foreshadow many of the principles seen in modern fractal theory. The concept of harmony and proportion, especially as expressed through the Golden Ratio, was deeply embedded in their philosophical and architectural practices. Euclid’s *Elements*, for example, constructed a rigorous geometric framework from basic postulates, an approach resonant with the recursive construction principles that define many fractals today [NA22, Fed88].

The Golden Ratio, often associated with natural beauty and structural balance, appears repeatedly in both historical and natural contexts. Pythagorean scholars embedded this proportion into their symbols, such as the pentagram, and later thinkers like Phidias and Euclid explored its aesthetic and mathematical implications. This proportional reasoning, while not yet formalized into what we now call fractals, laid the groundwork for perceiving self-similarity and recursive patterns as meaningful structures in mathematics and art [NA22, Add97].

As mathematical knowledge passed from classical antiquity through the Islamic Golden Age and into the European Renaissance, these concepts evolved. Scholars from Arab and Indian traditions significantly advanced algebra and geometry, developing numerical systems and astronomical tools that later influenced European thinkers. One key figure in this transmission was Leonardo of Pisa, better known as Fibonacci, whose *Liber Abaci* introduced the Indian-Arabic numeral system and

famously posed the rabbit population problem, leading to the Fibonacci sequence. The convergence of this sequence toward the Golden Ratio further strengthened its mathematical relevance and natural occurrence [NA22].

By the late Middle Ages and into the Renaissance, figures such as Luca Pacioli and Leonardo da Vinci embraced the study of proportions not only as mathematical curiosity but as a lens to understand nature and art. Pacioli's *De Divina Proportione* (On the Divine Proportion) demonstrates this fusion of geometry and aesthetics, and even though the language of fractals had not yet emerged, the intuitive recognition of complex patterns within simple rules was already present. Artists and scholars began to appreciate that natural forms, leaves, coastlines, and even clouds, were governed by patterns that defied the traditional Euclidean geometry [CS10].

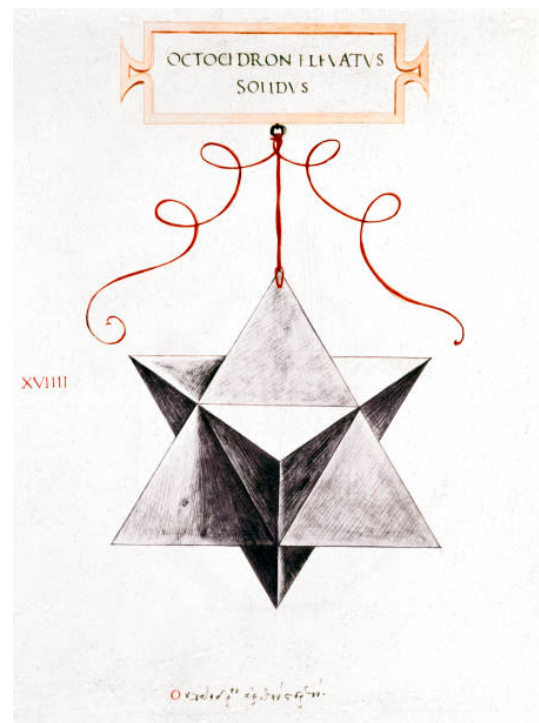


Figure 2.1: Drawings from the book *De Divina Proportione* [Pac09].

The true mathematical formalization of these ideas, however, only came much later. In the late 19th and early 20th centuries, mathematicians such as Karl Weierstrass, Georg Cantor, and Giuseppe Peano constructed bizarre functions and curves that challenged conventional understanding, continuous but nowhere differentiable functions, and space-filling curves, for example. These objects were initially regarded as pathological, but they eventually helped lay the theoretical groundwork for fractal geometry.

Finally, it was Benoît Mandelbrot who united these historical threads under the concept of fractals. In his seminal work, *The Fractal Geometry of Nature* (1982), Mandelbrot offered a new lens to describe natural irregularities mathematically. By

coining the term fractal, from the Latin *fractus* meaning “broken” or “fractured”, he acknowledged both the complexity and the self-similar elegance of the shapes found in natural phenomena. Fractals thus emerged not as a mathematical oddity but as a powerful framework to describe complexity across scales, from galaxy clusters to snowflakes [Fed88].

2.2 Mathematical Foundations

2.2.1 Self-Similarity and Recursion

A defining characteristic of fractals is **self-similarity**, which means that a fractal object contains smaller copies of itself at different scales. This property can be exact or approximate. Exact self-similarity occurs when the smaller parts are identical to the whole, as in mathematical constructs like the Cantor set or the Sierpinski triangle. In natural fractals, such as coastlines or clouds, the self-similarity is statistical rather than exact, meaning that the patterns repeat in a similar but not identical way [PJS04].

The generation of fractals is closely linked to **recursive** or iterative processes. Starting from a simple initial shape, a recursive rule is applied repeatedly to transform the shape into increasingly complex structures. This iterative nature explains how fractals can exhibit infinite detail and complexity, despite being described by relatively simple rules. For example, the Koch snowflake is created by recursively replacing each line segment with a pattern of four smaller segments, leading to a figure with infinite perimeter but finite area [PJS04].

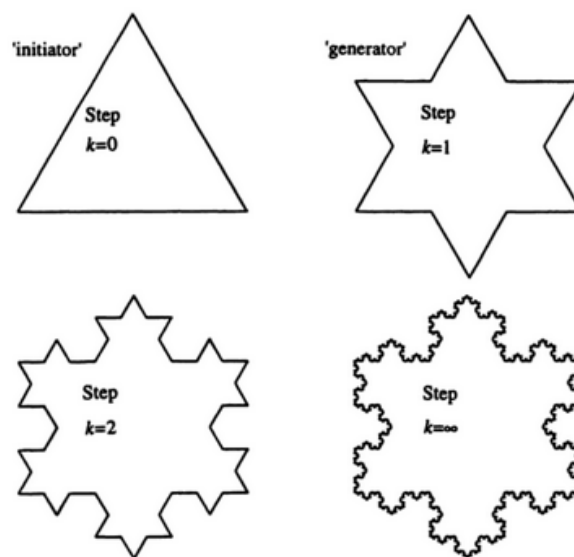


Figure 2.2: Step-by-step construction of the Koch snowflake, starting from an equilateral triangle [Add97].

The concept of recursion connects fractals to various mathematical fields such as dynamical systems and algorithm theory. It also provides a framework to model natural phenomena, where recursive growth processes are common, such as branching in trees or river networks. This recursive self-similarity is what makes fractals both fascinating and applicable across scientific disciplines.

2.2.2 Fractal Dimension

Unlike classical geometric shapes, fractals cannot be adequately described by integer dimensions. The **fractal dimension** is a measure that quantifies the complexity and space-filling capacity of fractal objects. It often takes non-integer values, reflecting the fact that fractals are more intricate than one-dimensional lines but do not completely fill two-dimensional space [PJS04].

One common way to estimate the fractal dimension is through the **box-counting method**. This involves covering the fractal with a grid of boxes of side length ϵ , then counting the number of boxes $N(\epsilon)$ that contain part of the fractal. As the box size decreases, the scaling behavior follows the relation:

$$N(\epsilon) \sim \epsilon^{-D}$$

where D is the fractal dimension. By plotting $\log N(\epsilon)$ versus $\log(1/\epsilon)$, the fractal dimension can be estimated from the slope of the linear region.

The fractal dimension captures how detail in the fractal changes with scale and is essential for comparing fractals and understanding their physical properties. For example, the Koch snowflake has a fractal dimension of approximately 1.26, indicating it is more complex than a line but less than a plane. Fractal dimensions have applications in physics, biology, and finance, wherever complex, scale-invariant patterns arise [PJS04].

2.3 Types of Fractals

Fractals can be broadly categorized based on their method of construction, mathematical properties, and resemblance to natural forms. This section presents three primary types: geometric fractals, complex fractals, and random (or natural) fractals. Each of these categories reveals a different facet of self-similarity, complexity, and mathematical elegance.

2.3.1 Geometric Fractals

Geometric fractals are constructed using simple, often recursive, geometric rules. They exhibit exact self-similarity, meaning each part of the fractal is an identical, scaled-down copy of the whole.

A classic example is the **Koch snowflake**, generated by iteratively modifying an equilateral triangle. Another is the **Sierpiński triangle**, formed by recursively removing the central triangle from a larger one. These fractals are typically defined by a deterministic algorithm and are easy to describe with formal rules.

Many geometric fractals can be described using Iterated Function Systems (IFS), a method where a finite set of contraction mappings is repeatedly applied. Despite their mathematical simplicity, such fractals often reveal surprisingly intricate structures.

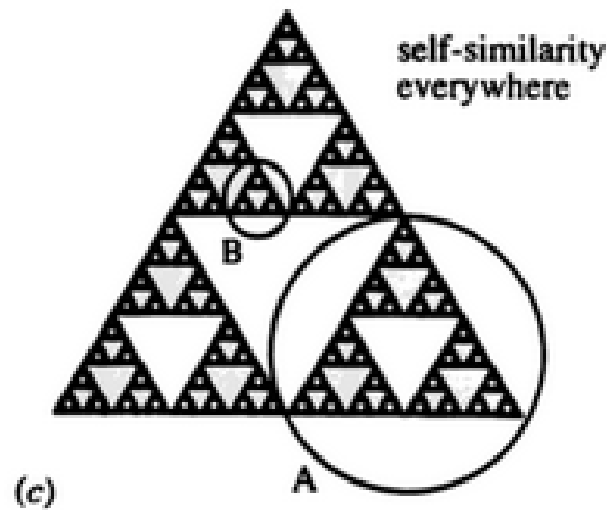


Figure 2.3: The Sierpiński triangle [Add97].

2.3.2 Complex Fractals

Complex fractals emerge from functions defined over the complex plane. They are typically visualized by iterating a complex function and analyzing the behavior of the resulting sequence for each point $z_0 \in \mathbb{C}$.

The most famous example is the **Mandelbrot set**, defined by the recurrence relation:

$$z_{n+1} = z_n^2 + c, \quad z_0 = 0$$

for complex numbers c . The set contains all c for which the sequence remains bounded. Its boundary is infinitely intricate and exhibits self-similarity at various

scales.

Another important example is the family of **Julia sets**, where the same recurrence is used, but with fixed c and varying z_0 . Depending on the value of c , the resulting set can be connected or totally disconnected.

These fractals highlight the deep connection between fractal geometry and complex dynamics.

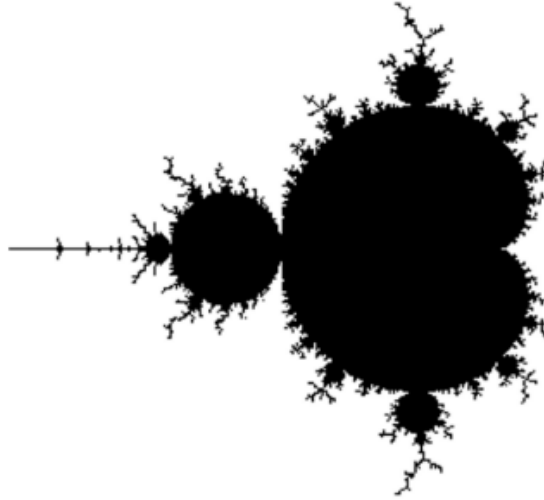


Figure 2.4: The Mandelbrot Set [Add97].

2.3.3 Random or Natural Fractals

Unlike geometric or complex fractals, natural fractals often display statistical rather than exact self-similarity. These include objects found in nature such as coastlines, mountains, trees, lightning bolts, and clouds.

Mathematically, they are often modeled using stochastic processes such as fractional Brownian motion, percolation theory, or random midpoint displacement. Their complexity and irregularity closely mirror the unpredictable patterns of nature.

A well-known model is the **fractal Brownian motion** (fBm), which extends classical Brownian motion by introducing memory and self-similarity. The Hurst exponent H governs the roughness of the generated structure.

These models have practical applications in physics, geology, computer graphics, and even finance, where fractal models are used to simulate market behavior.

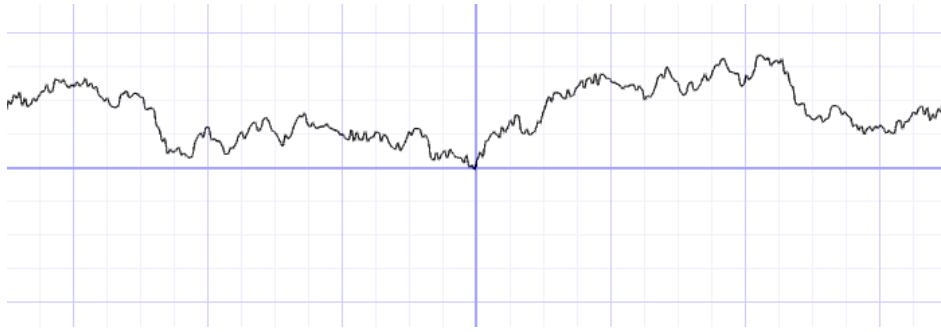


Figure 2.5: 2D fractal Brownian motion (fBm) visualization, resembling mountainous terrain.

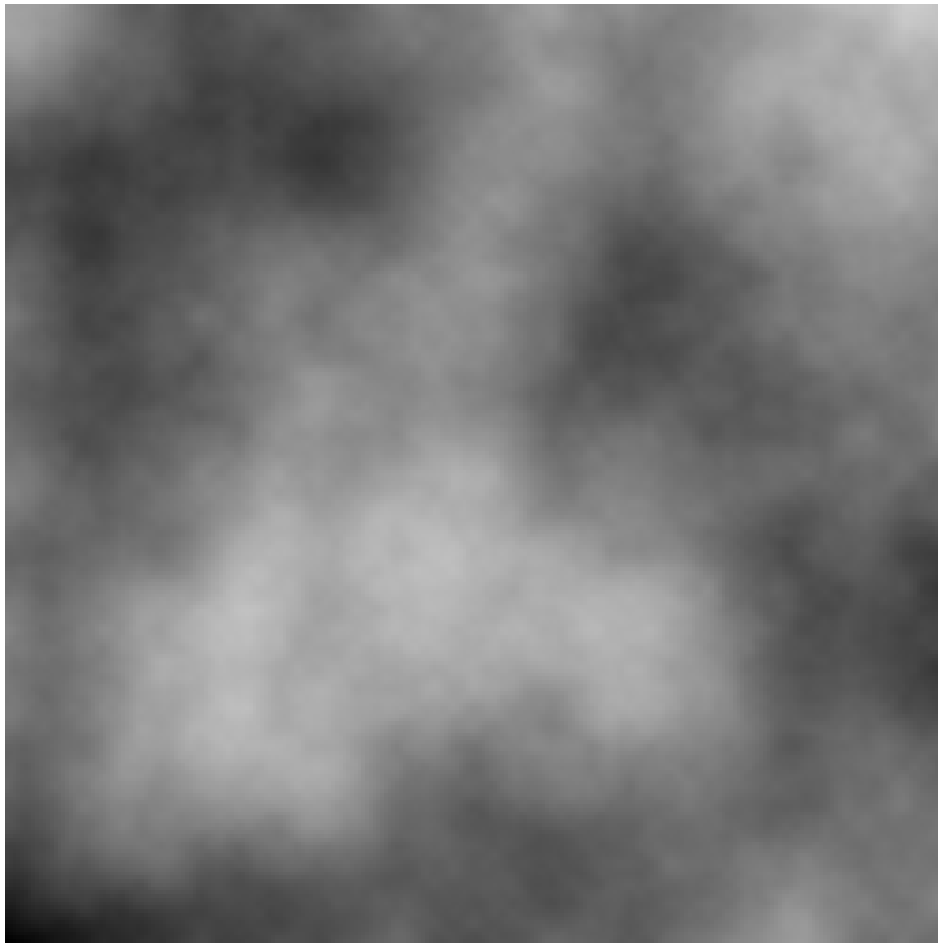


Figure 2.6: 3D fractal Brownian motion (fBm) visualization, resembling volumetric fog.

Both images were generated using the interactive tools from [\[VL15\]](#).

2.4 Applications of Fractals

2.4.1 Computer Graphics and Visualization

Fractals have had a profound impact on computer graphics and visualization, providing powerful tools to generate complex, natural-looking images with relatively simple algorithms. Their inherent self-similarity and recursive nature make them ideal for modeling natural phenomena such as mountains, clouds, coastlines, and trees, which are difficult to replicate using traditional Euclidean geometry. Techniques based on fractals, such as the Mandelbrot set, Julia sets, and iterated function systems (IFS), have enabled artists and programmers to create visually stunning and mathematically precise graphics that exhibit infinite detail at different scales [Fed88, NA22].

One major advantage of fractal-based graphics is their procedural generation capability, which allows for the creation of vast and detailed landscapes or textures without the need for storing large amounts of data. This has been especially useful in video games, simulations, and virtual reality environments where realistic natural scenes are required. Fractal algorithms can also produce realistic textures such as bark, rock surfaces, and clouds, improving the immersion and visual richness of digital content [PJS04].

Moreover, fractals are employed in rendering techniques to enhance visual effects like terrain roughness, water waves, and fog, all of which exhibit fractal-like properties. The ability to control parameters within fractal equations allows for diverse outcomes, enabling designers to tailor complexity and randomness to suit artistic or practical needs. This blend of mathematical rigor and creative flexibility has cemented fractals as an essential element in modern computer graphics [PJS04].

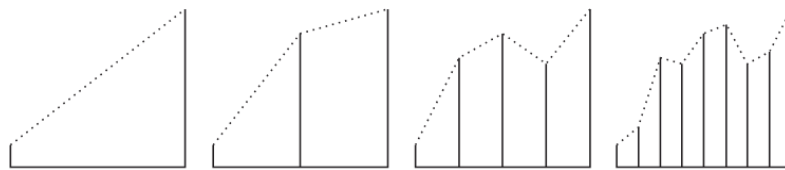


Figure 2.7: A random midpoint replacement rule [GS10].



Figure 2.8: Result of replacement rule in Figure 2.7 [GS10].

2.4.2 Natural Sciences

Fractal geometry has become a fundamental tool in various natural sciences for describing complex, irregular shapes and phenomena that defy traditional Euclidean geometry. Many structures and processes in nature exhibit fractal-like patterns, characterized by self-similarity and scale invariance, which can be effectively analyzed and modeled using fractal mathematics.

In biology, fractals appear in diverse contexts, from the branching patterns of trees and blood vessels to the intricate structure of lungs and neural networks. These fractal patterns optimize functions such as nutrient transport, surface area maximization, and signal transmission [Fed88, NA22]. For example, the bronchial tree in human lungs demonstrates recursive branching that can be quantified with fractal dimensions, reflecting an efficient design for gas exchange.

Geology and earth sciences also benefit from fractal analysis to understand complex phenomena such as coastline shapes, mountain ranges, river networks, and fault lines. The fractal nature of coastlines, first popularized by Mandelbrot, illustrates how measurements of length depend on the scale of observation, revealing the self-similar irregularity inherent in natural landscapes. Similarly, fractal models help characterize spatial patterns of earthquakes and mineral deposits.

In physics, fractals describe phenomena ranging from the distribution of galaxies in the universe to patterns in turbulent fluids and phase transitions. The fractal dimension quantifies irregularities and complexity in phenomena like diffusion-limited aggregation or the percolation clusters in disordered systems [Fed88]. These fractal structures often emerge spontaneously from nonlinear dynamical systems, highlighting the interplay between chaos and order in natural processes.

By applying fractal geometry, scientists can move beyond oversimplified models and better capture the complexity inherent in natural forms and phenomena, thus improving understanding, prediction, and simulation capabilities across disciplines.



Figure 2.9: Flower fractal generation in stages [GS10].

2.4.3 Signal Processing and Data Compression

Fractals have found significant applications in signal processing and data compression due to their ability to represent complex, self-similar structures efficiently. Many natural signals, such as audio, images, and even biological data, exhibit fractal-like patterns, which can be exploited to achieve better compression ratios without losing critical detail.

One of the key approaches is fractal image compression, which leverages the self-similarity present within images. By identifying smaller parts of an image that resemble larger sections, fractal compression encodes the image using transformation parameters rather than pixel-by-pixel data. This method allows high compression ratios and can reproduce images with good quality upon decompression [GS10].

In signal processing, fractal analysis helps characterize complex signals that traditional Fourier or wavelet methods might inadequately describe. For instance, fractal dimension and other fractal metrics can quantify the roughness or complexity of a signal, which is useful in biomedical signal analysis, geophysics, and telecommunications [Fed88].

Moreover, fractal-based algorithms are being explored for efficient encoding of audio and video streams, as well as in texture synthesis and pattern recognition. The recursive nature of fractals matches well with multi-scale analysis, enabling algorithms that are both computationally efficient and capable of capturing fine details across scales.

Despite some limitations, such as computational complexity during encoding, fractals remain a promising area for ongoing research and practical applications in signal compression and processing.

2.4.4 Fractals in Art and Popular Culture

Fractals have inspired artists, designers, and popular culture in various creative domains, extending beyond their mathematical and scientific origins. The intricate patterns and self-similarity inherent in fractals provide a rich visual language that resonates with both natural aesthetics and abstract art.

In the realm of visual arts, fractal geometry has been used to generate captivating digital artworks that evoke the complexity of nature. Artists employ fractal algorithms to create shapes and textures resembling natural objects such as trees, clouds, mountains, and coastlines, often pushing the boundaries of computer-generated art [GS10]. The book *The Beauty of Fractals* showcases numerous examples where fractals serve as a source of inspiration for both technical exploration and artistic expression.



Figure 2.10: "Fractal Mountains", by Anne Burns [GS10].

Fractals have also penetrated popular culture through media, films, and music. Movies such as *Jurassic Park* and *The Matrix* have featured fractal landscapes and effects, popularizing the concept to broad audiences. Musicians and composers have experimented with fractal patterns to structure compositions, reflecting the recursive and self-similar nature of fractals in sound [Add97].

Moreover, fractals appear in architectural designs and fashion, where repeating motifs and scaling patterns evoke a sense of harmony and complexity. This crossover highlights the broad cultural impact of fractal concepts, bridging science and creativity.

The fascination with fractals continues to grow as advances in computing enable ever more complex fractal visualizations, making fractal art a dynamic and evolving field within contemporary culture.

Chapter 3

GPU Computing and Fractals: First Implementation

3.1 Introduction to GPU Computing

GPUs, standing for Graphics Processing Units, are specialized hardware designed primarily for rendering graphics, but over the years they have evolved from helping CPUs in offloading the work to powerful parallel computing devices. Where Central Processing Units (CPUs) are good for sequential task execution, GPUs excel at simultaneous tasks. This makes them particularly suited for applications that require a lot more computational power than the CPU can provide.

Owing to their highly parallel architecture, GPUs are particularly well-suited for problems that can be decomposed into numerous independent computations. This characteristic has led to their successful application beyond graphics, especially in addressing embarrassingly parallel problems.

The ability of GPUs to process lots of threads concurrently allows them to perform computations at a much higher output, compared to the traditional CPUs, especially in computation-heavy domains such as scientific simulations and image processing.

3.1.1 Why Use the GPU?

The primary reason for using the GPU in different tasks, and in our case fractal generation, is its inherent parallel processing advantages. Unlike CPUs, which generally have a smaller number of cores and are more general purpose in processing, GPUs are equipped with hundreds to thousands of smaller cores, more capable for concurrent tasks. This is the main advantage of them in this tasks that can be broken down easily.

We can take as an example our topic on fractal generation. Since fractals require

a high number of iterations of mathematical functions across a grid, GPUs can handle these operations more efficiently in parallel, significantly speeding up the time compared to CPU-based solutions.

In addition to their raw computational power, modern GPUs also offer specialized features, including:

- High-speed memory such as Graphics Double Data Rate (GDDR), similar to DDRs but specifically designed for GPUs and high-performance computing.
- Wide range of libraries and development platforms, such as NVIDIA's CUDA or OpenCL, bringing well made tools and architectures for the users.
- Continuous support and hardware innovation aimed at general-purpose GPU computing, improving the current capabilities and upgrading for the future of widely used tasks like AI.

3.1.2 Advantages Over CPU for Fractal Generation

Fractal generation, much like ray-tracing or other types of graphics rendering, involves intensive computation of mathematical calculations over a given space. At its core, this process requires performing recursive or iterative calculations on the points of the said given space. Because of this complex mathematical problems and variables like the dimension of the space grid that can get larger and larger, executing the generation on CPUs can result in long computation times for the results. However, the GPU is well-suited for this kind of task.

One key advantage of using a GPU for fractals generation is the parallelism it offers. When rendering a fractal, we can process independently each pixel of it. This means that for each one of these pixels, the GPU can perform the required calculations in parallel, speeding up the generation process dramatically, making the change from CPU to GPU very noticeable.

Moreover, GPUs excel at handling large datasets. When generating complex fractals like the Mandelbrot or Julia sets, the number of points to process can be very large. GPUs can efficiently manage these datasets due to their large memory bandwidth and architecture optimized for such tasks, while CPUs may struggle under the same load.

Another advantage is the use of specialized GPU programming frameworks, such as CUDA (Compute Unified Device Architecture), which allows developers to write programs that can exploit the full potential of the GPU. With CUDA, developers can offload computationally intensive portions of the fractal generation process onto the GPU, achieving significant performance improvements over CPU-based implementations.

Overall, the combination of parallelism, high memory bandwidth, and specialized hardware support makes the GPU an ideal choice for fractal generation, offering substantial improvements in both speed and efficiency compared to traditional CPU-based approaches.

3.2 CUDA: Overview and Setup

3.2.1 Experimental Setup

All benchmarks and comparisons presented in this chapter were conducted on a desktop system with the following configuration:

- **CPU:** AMD Ryzen 5 3600X, 6 cores / 12 threads, base frequency 3.8 GHz
- **GPU:** MSI GeForce RTX 2060 SUPER VENTUS GP OC, 8 GB GDDR6
- **RAM:** 32 GB Excelem Dark DDR4, 3200 MHz, CL16 (dual-channel)
- **Motherboard:** ASRock B450M PRO4-F
- **Operating System:** Windows 10 Pro
- **CUDA Toolkit:** Version 12.6
- **Development Environment:** Visual Studio 2022, Version 17.12.3, 64-bit Release configuration

To ensure consistency and minimize variance, each benchmark was run approximately five times, and the average execution time was recorded.

3.2.2 Installation and Environment

To begin with CUDA programming, it is essential to properly install the necessary tools and set up the development environment. CUDA, developed by NVIDIA, is a parallel computing platform and programming model that allows developers to harness the power of GPUs for general-purpose computing. Setting up CUDA requires installing several components, including the CUDA Toolkit, GPU drivers, and a compatible IDE for development.

First, you need to install the NVIDIA driver for your GPU to ensure that your system can communicate effectively with the hardware. Next, you should install the CUDA Toolkit, which provides libraries, compilers, and tools necessary to write, compile, and execute CUDA programs. It's important to ensure that the toolkit version matches the installed GPU driver for compatibility.

In addition, CUDA supports a variety of programming languages, including C, C++, and Python, and integrates well with existing frameworks like OpenGL and DirectX for graphics. Once installed, you will also need to configure the environment variables, such as adding CUDA paths to the system's PATH and library variables, to allow for successful compilation and execution of CUDA code.

With the installation and environment setup complete, developers can begin writing GPU-accelerated code and testing their applications to ensure that the system is running properly.

3.2.3 CUDA Programming Model

The CUDA programming model is designed to make it easier for developers to write parallel programs that execute on the GPU. The model divides the computational work into two main components: the host (CPU) and the device (GPU). The host is responsible for managing the application and controlling the flow of execution, while the device performs the heavy lifting of the computation.

CUDA programs consist of two main types of functions: kernel functions and host functions. Kernel functions are executed on the GPU, and they are called from the host code. These kernels are executed in parallel across a grid of threads, where each thread works on a specific subset of the data.

One key concept in CUDA programming is thread hierarchy. Threads are organized into blocks, and blocks are organized into grids. This hierarchy enables efficient parallel execution, as threads within a block can cooperate and share data through shared memory, while blocks can be scheduled independently across different multiprocessors on the GPU.

In addition to managing thread execution, CUDA provides mechanisms for memory management. The host can allocate memory on the device, copy data from the host to the device and vice versa, and synchronize data when necessary. Understanding the CUDA programming model is critical for efficiently utilizing the power of GPUs for computationally intensive tasks like fractal generation.

3.3 The First Fractal: Julia Set

3.3.1 Mathematical Definition

The Julia Set is a well-known fractal that is generated by iterating a simple complex function of the form

$$f(z) = z^2 + c$$

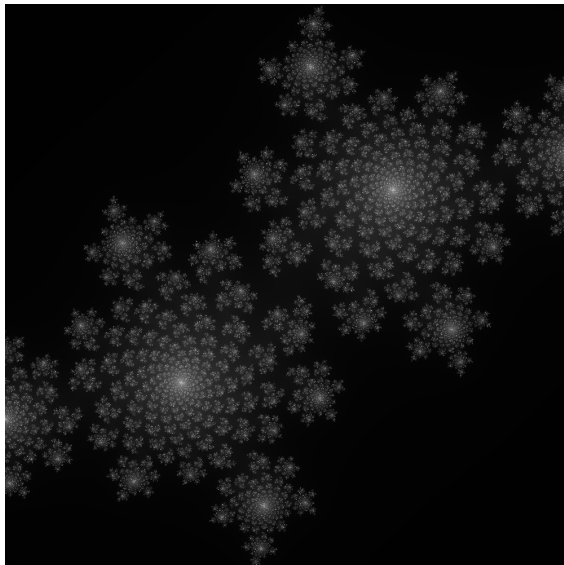
(*Quadratic polynomial*), where z is a complex number and c is a constant complex parameter. The set is formed by examining the behavior of the sequence generated by iterating this function for each point in the complex plane.

For each point z_0 in the complex plane, the sequence

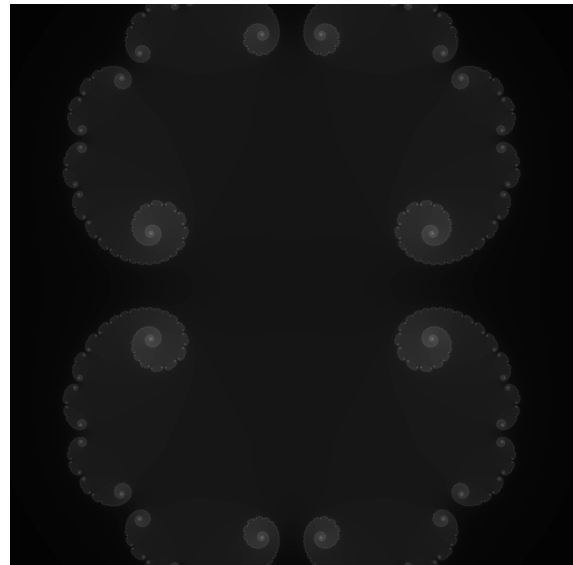
$$z_1 = f(z_0), \quad z_2 = f(z_1), \quad z_3 = f(z_2), \dots$$

is computed. The behavior of this sequence determines whether the point belongs to the Julia Set. If the sequence remains bounded (i.e., the magnitude of the complex numbers does not grow indefinitely), then the point is considered part of the Julia Set; otherwise, it is not.

The boundary of the Julia Set exhibits intricate, self-similar patterns. The structure of the set can vary dramatically depending on the value of the constant c . For different values of c , the Julia Set can look entirely different—ranging from connected and smooth to fragmented and chaotic. The Julia Set is also closely related to the Mandelbrot Set, with each point in the Mandelbrot Set corresponding to a unique Julia Set.



(a) Julia Set with $c = -0.4 + 0.6i$



(b) Julia Set with $c = 0.285 + 0i$

Figure 3.1: Examples of Julia Sets for different values of c

3.3.2 Why the Julia Set Was Chosen

The Julia Set was chosen for this project as implemented fractal for several reasons. Firstly, it provides a simple mathematical definition that can be easy to understand, explain and efficiently computed. The iterative nature of the function allows for a very straightforward parallelization, which is ideal for GPU acceleration. The

fact that each point in the plane can be independently processed makes it a very good choice.

Moreover, the Julia Set's visual appeal with its interesting shapes and the diverse fractals from changing one value in the mathematical formula makes it a fun and great starting point for exploring fractals. The complex variable c being the key factor influencing the shape of the set allows for extensive experimentation. The Julia Set serves as a natural introduction to GPU programming for fractals. It's a well-defined structure and relatively simple in computation, allowing us to focus on implementation, understanding and future optimization of the code.

Another suitable choice would have been the Mandelbrot Set, which is closely related one to the Julia Set and similarly defined by a simple iterative formula. However, the Julia Set was chosen because a key difference lies in their approaches. The Mandelbrot Set is generated by iterating the point $z = 0$ for different values of the complex parameter c , effectively exploring the parameter space. In contrast, the Julia Set fixes c and iterates over each point in the plane. Although the Mandelbrot Set is more visually iconic, the Julia Set provides greater flexibility in exploring how the complex variable c directly affect the generation of the fractal, allowing better exploration for the user.

3.4 Code Architecture

3.4.1 Main Components

For our task to generate the Julia Set, we will build it using CUDA in order to take full advantage of GPU parallelism. At its base, the rendering process involves iterating the mathematical formula over many points in the complex plane, a fitting task for multiple threads running in parallel simultaneously. The initial implementation used in this chapter is adapted from the example provided in *CUDA by Example* by Sanders and Kandrot [SK10], which serves as a foundation for further enhancements and optimizations.

Firstly, we will need to prepare some functions, and we will start with defining a structure for complex numbers. CUDA doesn't provide a complex number type natively, so we will have to implement our own custom structure, named `cuComplex`, which will handle basic arithmetic on complex numbers directly on the GPU. We will implement for multiplication and addition, along with a function to compute the square magnitude. These are the essential functions for our complex structure for evaluating whether a point escapes to infinity.

Noteworthy, CUDA provides two ways to use certain parts of our code directly on the GPU, the function qualifiers `__device__` and `__global__` which are placed be-

fore a function. These two qualifiers are somewhat similar, both telling the compiler that the respective function will be handled by the GPU. The difference between them is the use case:

- **Device**
 - Can only be called from other GPU functions (`__device__` or `__global__` functions).
 - Can return values like normal functions.
- **Global**
 - Can be called only from the CPU/host.
 - Cannot return a value.

This doesn't help just in the easier understanding of the code, but gives a sense of hierarchy, `__device__` great for helper functions and `__global__` serving as the entry point for launching GPU execution.

```
struct cuComplex {
    float r, i;

    __device__ cuComplex(float a, float b) : r(a), i(b) {}

    __device__ float magnitude2(void) {
        return r * r + i * i;
    }

    __device__ cuComplex operator*(const cuComplex& a) {
        return cuComplex(r * a.r - i * a.i, i * a.r + r * a.i);
    }

    __device__ cuComplex operator+(const cuComplex& a) {
        return cuComplex(r + a.r, i + a.i);
    }
};
```

Listing 3.1: cuComplex structure

We now can proceed with the implementation of the Julia Set function, where we will apply the escape-time algorithm. We first prepare the values needed for the calculations:

- `scale` – a scaling factor used to normalize pixel coordinates into the range of the complex plane, which primarily affects the visual output.
- `jx` and `jy` – represent the real and imaginary parts of the current point, computed based on the pixel's position and the scale factor.
- `c` and `a` – `c` is the fixed complex constant that defines the shape of the Julia Set, while `a` is the initial complex point being iterated.

With the variables done we will complete the function with a loop for the calculations. Inside the loop, we iteratively apply the function $f(z) = z^2 + c$ to the current point a . After each iteration, we check the squared magnitude of a to determine if it has “escaped”, meaning its value has grown beyond a predefined threshold (here, 1000). If it does, we stop iterating early, as the point is not part of the Julia Set anymore. Finally we return the number of iterations completed, to be later used to calculate the color of the corresponding pixel, providing the fractal's distinctive gradient and structure.

```
__device__ int julia(int x, int y) {
    const float scale = 1.5;
    float jx = scale * (float)(DIM / 2 - x) / (DIM / 2);
    float jy = scale * (float)(DIM / 2 - y) / (DIM / 2);

    cuComplex c(-0.8, 0.156);
    cuComplex a(jx, jy);

    int i = 0;
    for (i = 0; i < ITERATIONS; i++) {
        a = a * a + c;
        if (a.magnitude2() > 1000) {
            break;
        }
    }
    return i;
}
```

Listing 3.2: Julia function

The last function needed in our generation process is the kernel function, which will be responsible for launching the threads in a 2D grid, with each thread corresponding directly to a pixel in the output image. Each thread launched will have the task to compute the color of its assigned pixel based on the number of iterations returned by the `julia` function in the position `x` and `y`.

```

__global__ void kernel(unsigned char* ptr) {
    int x = blockIdx.x;
    int y = blockIdx.y;
    int offset = x + y * gridDim.x;

    int juliaValue = julia(x, y);

    unsigned char color = (unsigned char)(255 * (float)juliaValue /
    ITERATIONS);

    ptr[offset * 4 + 0] = color; // Red
    ptr[offset * 4 + 1] = color; // Green
    ptr[offset * 4 + 2] = color; // Blue
    ptr[offset * 4 + 3] = 255;   // Alpha
}

```

Listing 3.3: Kernel function

3.4.2 Memory Usage and Data Flow

To complete the implementation, the main function will be the one to orchestrate the overall execution flow of the program. It will be responsible for initializing the environment, allocating the necessary memory on both the host and GPU, launching the GPU kernel, and finally retrieving and displaying the generated fractal image. Before diving into the details of the main function, it's important to note the key libraries included and some fundamental constants defined at the top of the source file. The libraries prepare the groundwork for the CUDA execution and image handling, while the constants exist for an easier time in experimenting.

- **Includes:**

- `book.h` - A utility header provided in the CUDA by Example codebase, containing macros and helper functions to simplify CUDA error handling and debugging.
- `stdio.h` - Standard C library header for input/output operations, used here primarily for console printing.
- `cpu_bitmap.h` - Provides the `CPUBitmap` class to facilitate creation, manipulation, and display of bitmap images on the CPU side.
- `cuda_runtime.h` - CUDA runtime API header that allows management of device memory, kernel launches, and other CUDA operations.

- **Defines**

- DIM - Specifies the width and height of the square output image, effectively setting the resolution of the rendered fractal.
- ITERATIONS - Determines the maximum number of iterations used in the escape-time algorithm in the `julia` function, controlling the fractal detail and computation workload.

```
#include "common/book.h"
#include <stdio.h>
#include "common/cpu_bitmap.h"
#include <cuda_runtime.h>

#define DIM 1000
#define ITERATIONS 500
```

Listing 3.4: Libraries and constants

The `main` function serves as the central orchestrator for the Julia Set generation program. First, it creates a `CPUBitmap` object to represent the output image on the host side (CPU), then allocates the necessary memory on the GPU using `cudaMalloc`. Next, it sets up a CUDA grid with dimensions matching the image resolution, ensuring that each thread corresponds to a single pixel.

The kernel is then launched, where each GPU thread computes the color for its assigned pixel based on the escape-time algorithm implemented in the `julia` function. After kernel execution, the resulting image data is copied back from the device to host memory using `cudaMemcpy`. Finally, the `CPUBitmap` object displays the rendered fractal image on the screen, providing immediate visual feedback.

This straightforward main function covers all the essential steps: resource initialization, GPU computation, and image presentation. It forms a solid foundation for any future enhancements or optimizations.

```

int main(void)
{
    CPUBitmap bitmap(DIM, DIM);
    unsigned char* dev_bitmap;

    HANDLE_ERROR(cudaMalloc((void**)&dev_bitmap, bitmap.image_size
    ()));

    dim3 grid(DIM, DIM);
    kernel << <grid, 1 >> > (dev_bitmap);
    printf("Kernel launched.\n");

    HANDLE_ERROR(cudaMemcpy(bitmap.get_ptr(), dev_bitmap, bitmap.
    image_size(), cudaMemcpyDeviceToHost));
    printf("Image data copied back to host.\n");

    cudaFree(dev_bitmap);

    // Display the bitmap and wait for user to close window
    bitmap.display_and_exit();

    return 0;
}

```

Listing 3.5: Libraries and constants

3.5 Initial Results

3.5.1 Output Quality

The first result from the GPU-accelerated Julia Set generator shows promising quality in terms of fractal's visual appearance. The output image exhibits the characteristics of fractals: intricate boundaries and self-similarity in patterns. The quality of the output is largely determined by the number of iterations performed and the resolution. Higher iteration counts and higher resolution lead to not just a bigger output, but a more detailed and smoother fractal, and the higher we go, the more we could observe little things like the self-similarity as we zoom into the output image.

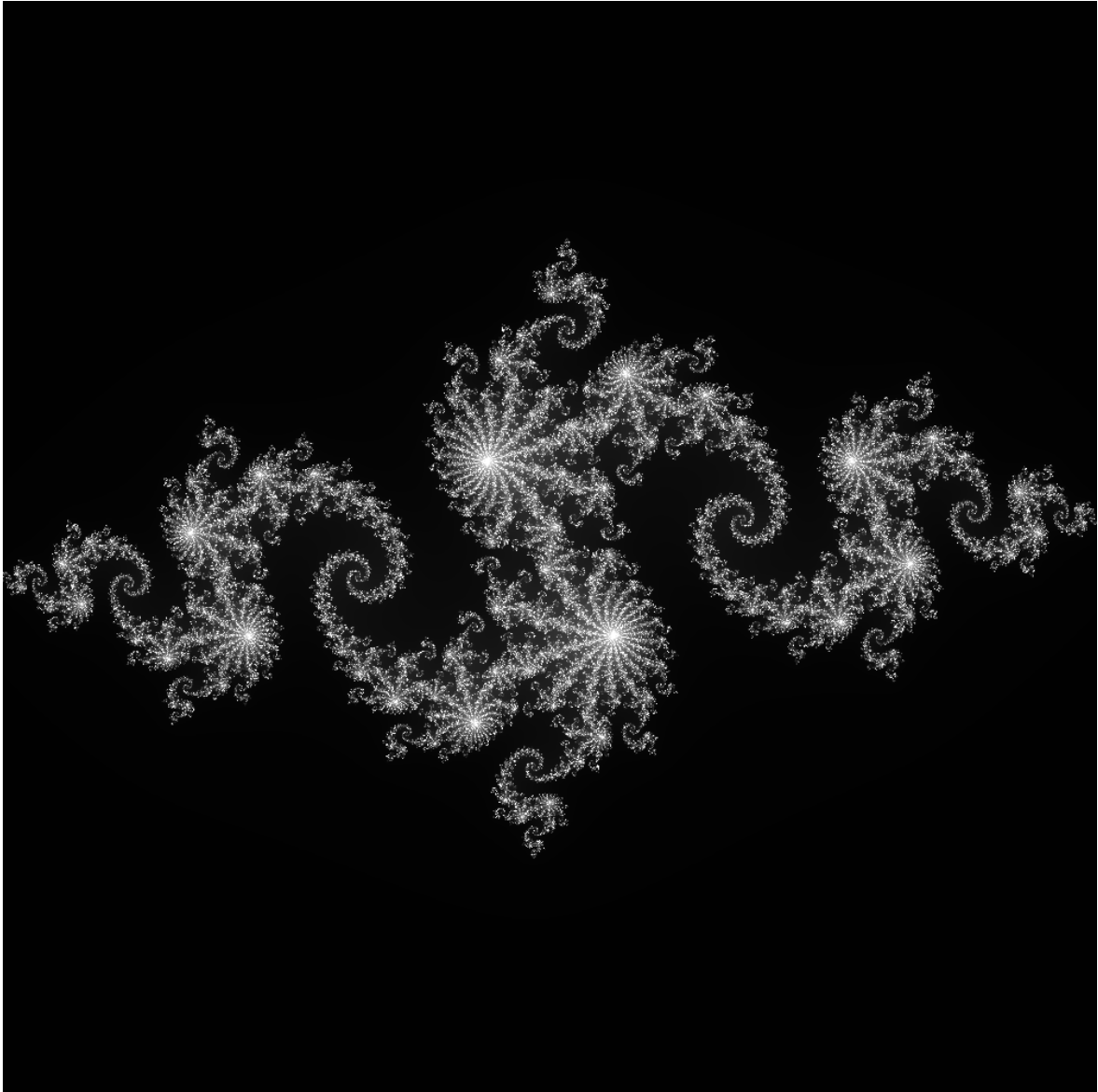


Figure 3.2: First version of the Julia Set generated on the GPU

Overall, the visual output successfully captures the complexity and beauty of the Julia Set, showing that the GPU implementation already achieves an impressive level of detail, even in this early stage. While the image quality is largely satisfactory and the time to render respectable, this initial version provides a solid base, suitable for future optimization strategies and additional changes based on preference, such as improved color mapping or refining the interface with more modern display approaches like OpenGL.

3.5.2 Performance Observations

To assess the benefits of GPU acceleration, a CPU-only version of the Julia Set generator was also implemented. This version uses a simple nested loop to compute the pixels sequentially on the host processor, following the same logic as the GPU kernel but without parallelism.

```
#include <stdio.h>
#include <cmath>
#include <chrono>
#include "common/cpu_bitmap.h"

#define DIM 1000
#define ITERATIONS 500

struct cuComplex {
    ...
};

int julia(int x, int y) {
    ...
}

int main() {
    CPUBitmap bitmap(DIM, DIM);
    unsigned char* ptr = bitmap.get_ptr();

    for (int y = 0; y < DIM; y++) {
        for (int x = 0; x < DIM; x++) {
            int offset = x + y * DIM;
            int juliaValue = julia(x, y);

            ptr[offset * 4 + 0] = juliaValue % 256;
            ptr[offset * 4 + 1] = (juliaValue * 2) % 256;
            ptr[offset * 4 + 2] = (juliaValue * 3) % 256;
            ptr[offset * 4 + 3] = 255;
        }
    }
    bitmap.display_and_exit();
    return 0;
}
```

Listing 3.6: CPU version of the Julia Set

Note: The `cuComplex` structure and `julia()` function used here are identical to those in the GPU version for consistency and comparability.

With the GPU and CPU versions now present, we conduct a series of tests using both implementations across varying resolutions and iterations.

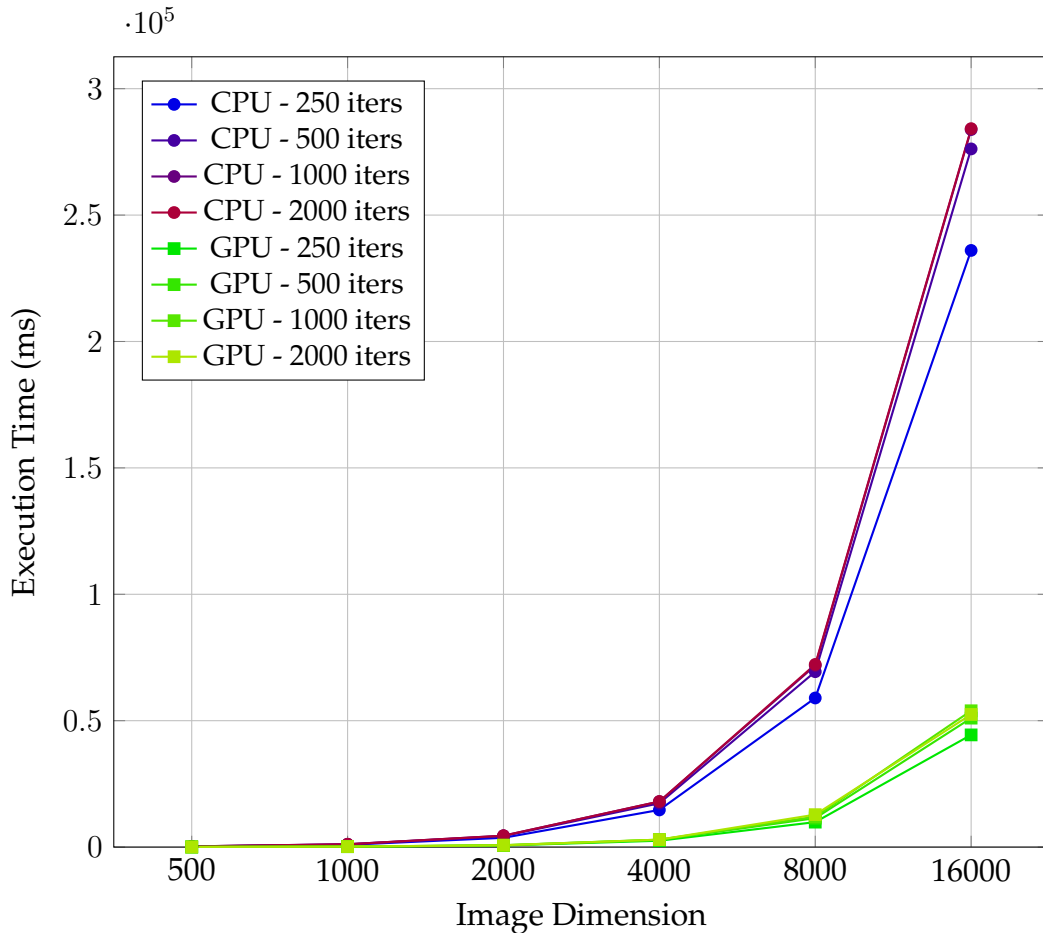


Figure 3.3: Execution time comparison between CPU and GPU implementations across different iteration counts

As expected, the GPU implementation consistently outperforms the CPU version across all tested dimensions and iteration counts. Thanks to its parallel architecture, the GPU processes thousands of pixels simultaneously, significantly reducing execution time compared to the sequential CPU approach. While the performance difference is minimal for smaller parameters, this performance advantage becomes especially pronounced at higher resolutions and iteration depths, demonstrating the clear benefits of GPU acceleration for fractal generation tasks.

3.6 Concluding Remarks

With the initial implementation of the Julia Set fractal generator complete and GPU acceleration successfully integrated, the results clearly highlight the performance gains over a CPU-only approach. This marks an important starting point for exploring more advanced techniques and optimization strategies in the next chapter.

The next chapter will dive into optimization techniques aimed primarily at improving execution speed and computational efficiency on the GPU. These optimizations will serve to enhance the responsiveness and scalability of the fractal generation, enabling exploration of even higher resolutions and iteration counts without heavy drawbacks in execution time.

Chapter 4

GPU Optimization Strategies for Fractal Rendering

4.1 Introduction and Optimization Motivation

Fractal rendering, especially for complex sets, is highly computational due to the repetitive nature of its escape-time algorithm. While the initial GPU implementation already demonstrated significant improvements over the CPU-based version, it was built using a relatively simple and unoptimized approach. While helpful for initial development and clarity, this simplicity also leaves room for optimization trials that could push performance further.

The main motivation for optimizing the GPU implementation stems from the desire to maximize computational efficiency, reduce execution times, and make the fractal generator more scalable for high-resolution images and high iteration counts. Additionally, refining the memory access patterns and thread configuration can help reveal the potential of modern GPUs and prepare the groundwork for possible interactive or real-time rendering features in the future.

This chapter explores several targeted optimization strategies, each addressing different architectural aspects of CUDA programming, and provides performance comparisons to illustrate their impact.

4.1.1 Overview of Implementation

In this version, each thread is responsible for computing a single pixel by iterating the Julia formula until a bailout condition is met or a maximum number of iterations is reached. The result is stored in global memory and eventually transferred to the host for visualization. A simple version, but during the testing phase, several architectural and design limitations are present, hindering its scalability and performance under heavier loads:

- **Global Memory Bottleneck:** All intermediate and final results are written directly to global memory, which incurs high latency.
- **Lack of Memory Coalescing:** Thread memory accesses are not optimized, leading to inefficient use of memory bandwidth.
- **No Use of Shared Memory:** All memory accesses are made directly to global memory, which is relatively slow. This leads to performance bottlenecks, especially when repeated access to the same data occurs across threads.
- **Rigid Thread Mapping and Pixel-Level Granularity:** Each thread is statically mapped to a single pixel using a fixed block and grid size. This approach lacks flexibility and misses opportunities for tuning performance through adjustable block sizes or work aggregation (e.g., processing multiple pixels per thread), especially at higher resolutions.

4.1.2 Why It Still Matters

Despite its simplicity, the baseline version is important because it establishes a clear and understandable foundation. It allows straightforward benchmarking of performance improvements in later versions, and helps isolate the impact of each optimization introduced. It also demonstrates how a naive parallel implementation can still outperform a CPU-bound version by leveraging raw parallelism.

4.2 Step-by-Step GPU Optimizations

With the baseline GPU implementation established, this section introduces a sequence of optimizations aimed at improving performance and scalability. Each version addresses specific limitations in memory access and thread configuration, building incrementally toward a more efficient rendering pipeline. These variants will be analyzed and compared later to assess their individual and impact.

4.2.1 Unified Memory Access

The first optimization introduced is the switch from traditional CUDA memory allocation with `cudaMalloc` to Unified Memory using `cudaMallocManaged`. This change simplifies memory management by allowing both the CPU (host) and GPU (device) to access the same memory space without the need for explicit memory transfers. Although this doesn't alter the core computation logic or execution structure, it improves code maintainability and reduces the potential for memory copy errors.

The modification is minimal, requiring only a single line of code to be updated. Below is a comparison between the original and the modified allocation lines:

```
unsigned char* dev_bitmap;
cudaMalloc((void*)&dev_bitmap, bitmap.image_size());
```

Listing 4.1: Original version using `cudaMalloc`

```
unsigned char* dev_bitmap;
cudaMallocManaged((void*)&dev_bitmap, bitmap.image_size());
```

Listing 4.2: Updated version using `cudaMallocManaged`

Unified Memory handles data migration between host and device automatically at runtime. This makes it easier to write correct CUDA code, particularly for beginners, and is often sufficient for programs that do not require highly optimized memory access patterns.

Additionally, the explicit memory copy from device to host using `cudaMemcpy` becomes optional after synchronization. Since Unified Memory allows the host to access the memory after `cudaDeviceSynchronize()`, the following line can be removed without affecting correctness:

```
// Optional when using Unified Memory
cudaMemcpy(bitmap.get_ptr(), dev_bitmap, bitmap.image_size(),
            cudaMemcpyDeviceToHost);
```

While the change to unified memory is minimal, it subtly improves safety by reducing the chances of memory mismanagement. Although it does not significantly alter the structure of the program, it enhances robustness and streamlines development. In some cases, unified memory may offer a slight performance boost by eliminating explicit memory copies and reducing transfer overhead, particularly for moderate image sizes where page migration costs are low. However, this often comes at the expense of overall performance, as the automatic data movement can introduce latency and inefficiencies. Thus, using unified memory represents a trade-off between improved safety and potential performance penalties.

4.2.2 Switch to Pitched Memory

To improve memory alignment and access efficiency, especially for 2D data like images, CUDA provides `cudaMallocPitch`, which allocates memory with padding to ensure optimal alignment. This version introduces pitched memory allocation in place of standard linear memory. While we also introduce a block and grid configuration, each thread still maps to exactly one pixel, keeping the execution model

unchanged. This block structure is required by CUDA's pitched memory mechanism but will be leveraged more fully in later optimizations.

The main changes in this version are:

- Replacing `cudaMallocManaged` with `cudaMallocPitch`, which returns both a pointer and a pitch (the row width in bytes, including padding).
- Using `cudaMemcpy2D` to copy memory from device to host, respecting the pitch.
- Adding a grid and block configuration to the kernel launch, although still assigning one thread per pixel, with no shared memory or intra-block cooperation yet.
- Slight updates in the kernel to calculate pixel positions and access rows using the pitch value.

We begin with changes inside the `main` function. The memory is now allocated using `cudaMallocPitch`, which ensures proper alignment for 2D memory access patterns. As a consequence, we also need to use `cudaMemcpy2D` when copying data back to the host. Additionally, we define a block and grid size suitable for the image dimensions, although each thread still operates independently, and the kernel does not yet leverage block-level parallelism.

```
int main(void)
{
    CPUBitmap bitmap(DIM, DIM);
    unsigned char* dev_bitmap;
    size_t pitch;

    // Allocate pitched memory
    cudaMallocPitch(&dev_bitmap, &pitch, DIM * 4, DIM);
    // Define block and grid sizes
    dim3 blockSize(16, 16);
    dim3 gridSize((DIM + 15) / 16, (DIM + 15) / 16);
    // Launch kernel
    kernel<<<gridSize, blockSize>>>(dev_bitmap, pitch);
    // Copy memory using cudaMemcpy2D
    cudaMemcpy2D(bitmap.get_ptr(), DIM * 4, dev_bitmap, pitch, DIM
    * 4, DIM, cudaMemcpyDeviceToHost);
    // Display and cleanup ...
}
```

Listing 4.3: Changes in `main()` for pitched memory

The kernel receives an extra parameter for the pitch and uses it to compute the row pointer for each pixel. This ensures each pixel write occurs at the correct memory location, accounting for alignment padding.

```
__global__ void kernel(unsigned char* ptr, size_t pitch)
{
    int x = threadIdx.x + blockIdx.x * blockDim.x;
    int y = threadIdx.y + blockIdx.y * blockDim.y;

    if (x < DIM && y < DIM)
    {
        unsigned char* row = (unsigned char*)((char*)ptr + y *
pitch);
        int offset = x * 4;

        int juliaValue = julia(x, y);
        unsigned char color = (unsigned char)(255 * (float)
juliaValue / ITERATIONS);

        row[offset + 0] = color;
        row[offset + 1] = color;
        row[offset + 2] = color;
        row[offset + 3] = 255;
    }
}
```

Listing 4.4: Adjusted kernel for pitched memory

Although block and thread indexing is introduced here, the program still effectively assigns one thread per pixel. There is no collaboration or data reuse within thread blocks, and no shared memory is utilized. The addition of the block and grid structure is primarily a syntactic requirement of launching kernels that use pitched memory efficiently, rather than an optimization by itself.

The core objective of this version is to improve the memory layout by using `cudaMallocPitch`, which aligns each row in device memory for more efficient memory access. This alignment minimizes potential issues such as misaligned accesses and memory bank conflicts, particularly important when processing large 2D data structures like images. While the computational pattern remains the same, this memory layout adjustment can lead to measurable performance benefits, especially as the resolution of the image increases.

4.2.3 Thread Block Configuration Improvement

In the previous version, we introduced pitched memory allocation to optimize global memory access. Although the program used `dim3` variables for block and grid dimensions, the launch configuration did not yet take full advantage of CUDA's hierarchical execution model. In this version, we configure thread blocks properly, enabling the GPU to distribute work across multiple threads per block more efficiently.

- A `BLOCKSIZE` macro is introduced to easily control the size of thread blocks.
- The block and grid dimensions are configured based on this macro.
- The kernel remains unchanged, as it already calculates the global coordinates correctly using thread and block indices.

The modified lines in the `main` function are as follows:

```
#define BLOCKSIZE 16

dim3 blockSize(BLOCKSIZE, BLOCKSIZE);
dim3 gridSize((DIM + BLOCKSIZE - 1) / BLOCKSIZE,
              (DIM + BLOCKSIZE - 1) / BLOCKSIZE);

kernel<<<gridSize, blockSize>>>(dev_bitmap, pitch);
```

Listing 4.5: Thread block configuration changes in `main`

This adjustment enables proper division of the workload. CUDA organizes the threads hierarchically: a kernel launch creates a grid of blocks, and each block contains multiple threads. In our 2D configuration, each thread computes a unique (x, y) pixel coordinate using its local thread indices (`threadIdx.x`, `threadIdx.y`) and the position of its block within the grid (`blockIdx.x`, `blockIdx.y`). These coordinates are used to access and modify the correct portion of memory corresponding to a single pixel.

By grouping threads into blocks, the GPU scheduler can more effectively distribute work and hide memory latency. Block sizes such as 16×16 (i.e., 256 threads per block) are chosen to align well with GPU hardware, particularly with warp execution and shared memory layouts. This configuration does not yet exploit shared memory or advanced memory coalescing strategies, but it represents a significant improvement in terms of occupancy and overall structure.

4.3 Comparative Analysis and Results

This section presents a detailed comparative analysis of the fractal rendering implementations, focusing on execution time benchmarks across varying image dimensions and iteration counts. Additionally, it explores the trade-offs between code complexity and performance optimizations achieved in each version, providing insight into their practical applications.

4.3.1 Execution Time Benchmarks

The following benchmarks illustrate the execution time performance of each fractal rendering implementation under varying image dimensions and iteration counts. Results are grouped to compare individual versions directly and to provide an overall summary using average execution times.

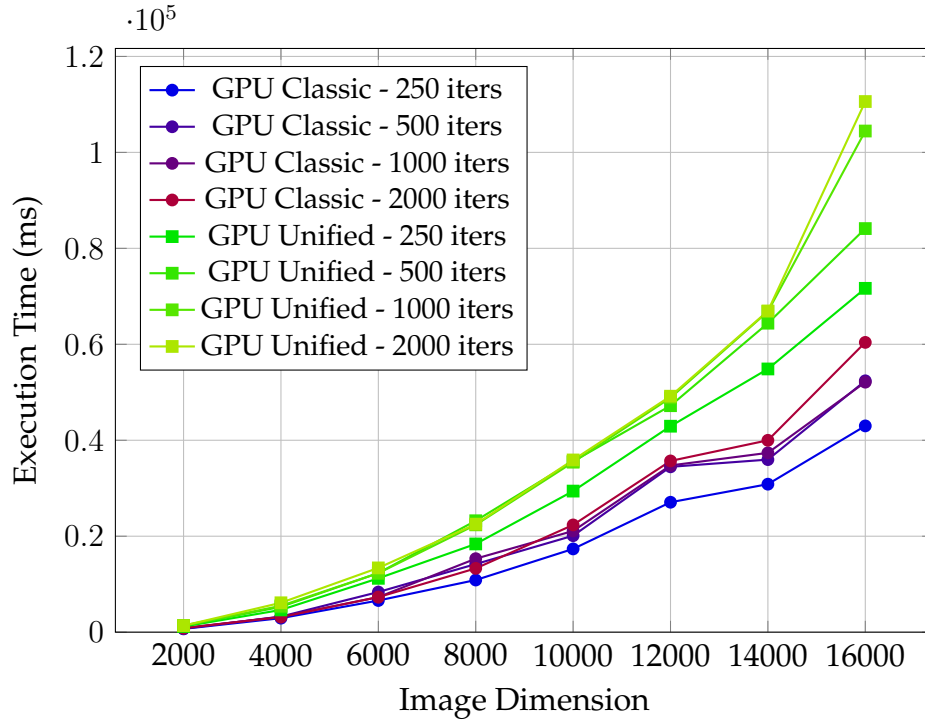


Figure 4.1: Execution time comparison between GPU Classic Memory and GPU Unified Memory for different iteration counts

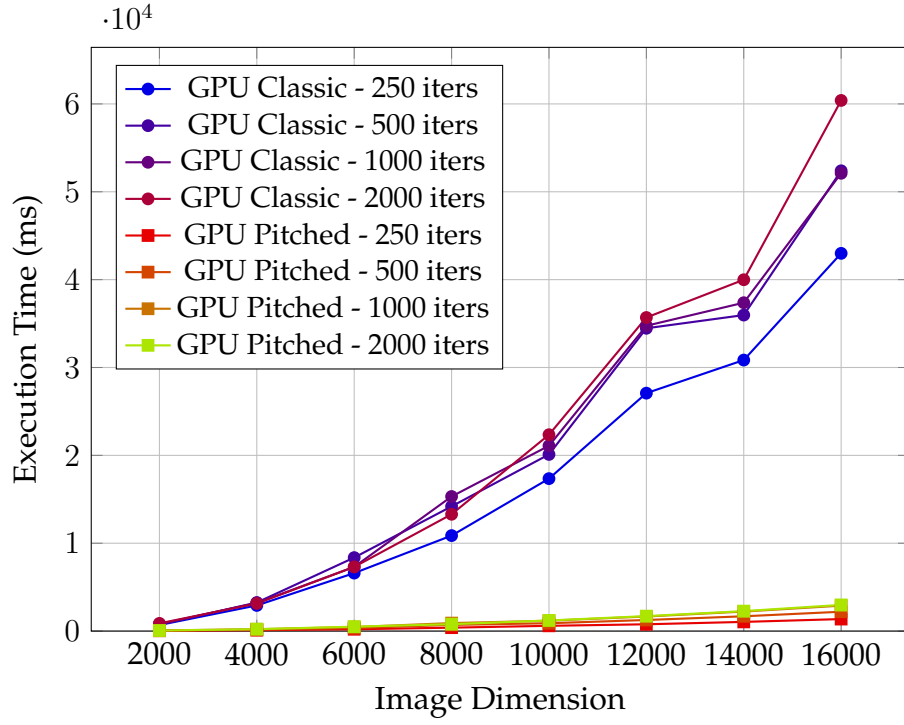


Figure 4.2: Execution time comparison between GPU Classic Memory and GPU Pitched Memory for different iteration counts

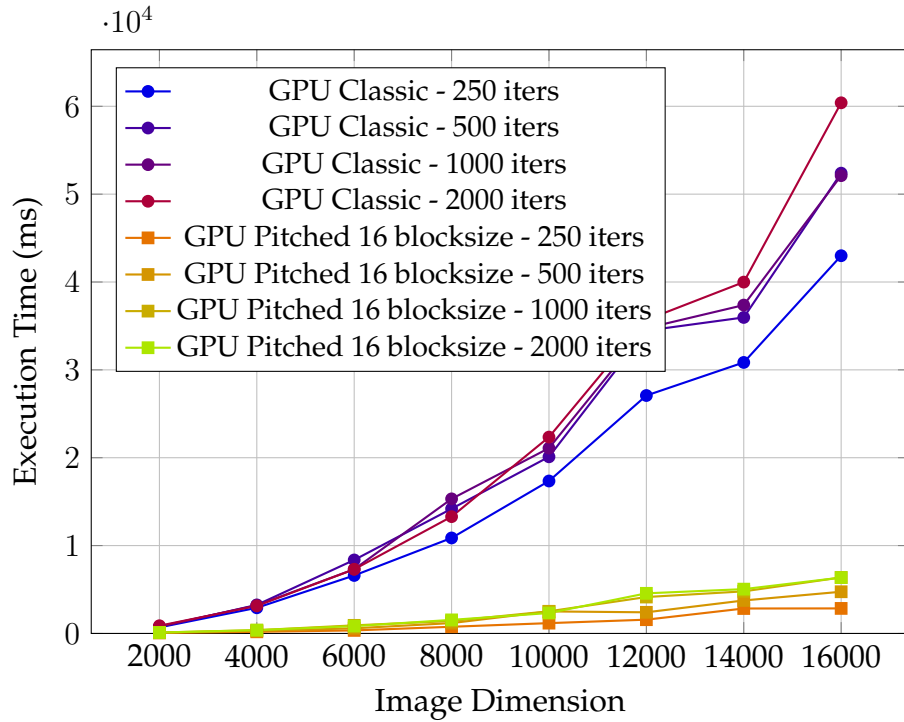


Figure 4.3: Execution time comparison between GPU Classic Memory and GPU Pitched Memory (16 blocksize) for different iteration counts

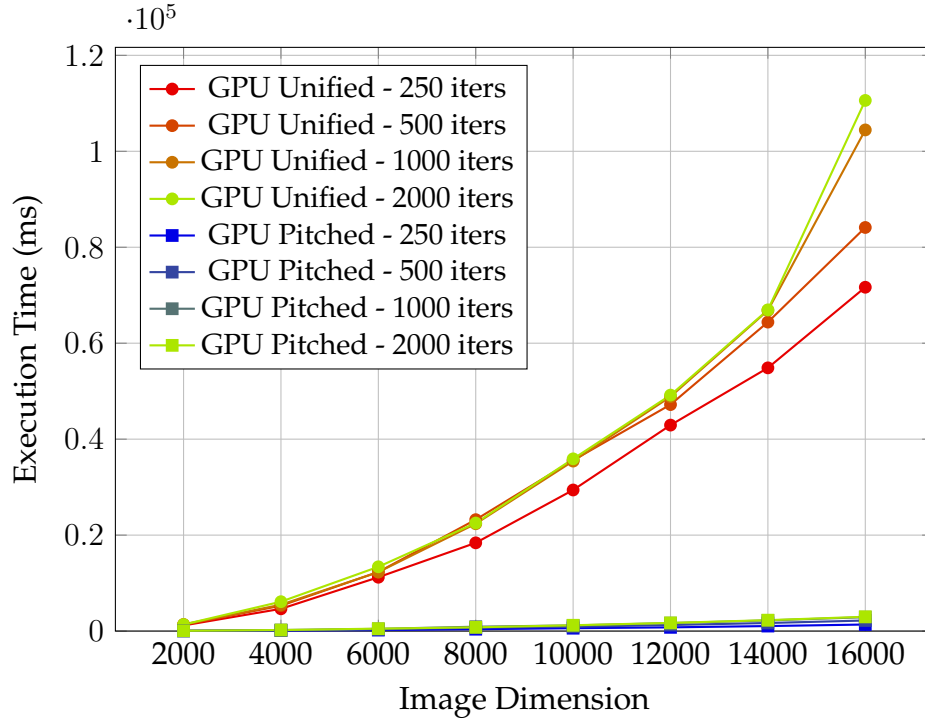


Figure 4.4: Execution time comparison between GPU Unified Memory and GPU Pitched Memory for different iteration counts

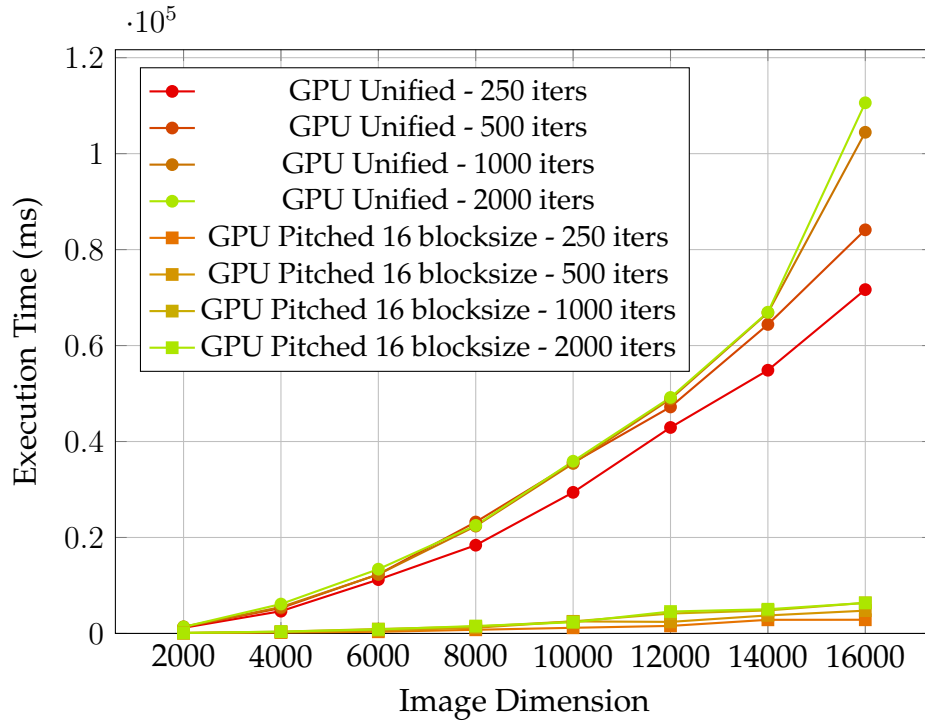


Figure 4.5: Execution time comparison between GPU Unified Memory and GPU Pitched Memory (16 blocksize) for different iteration counts

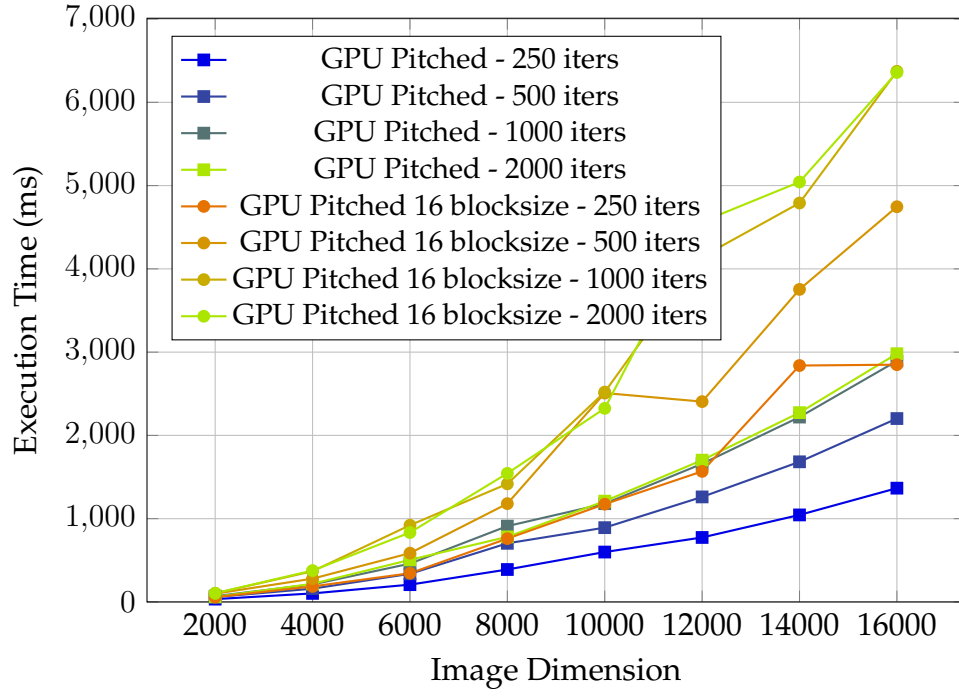


Figure 4.6: Execution time comparison between GPU Pitched Memory and GPU Pitched Memory (16 blocksize) for different iteration counts

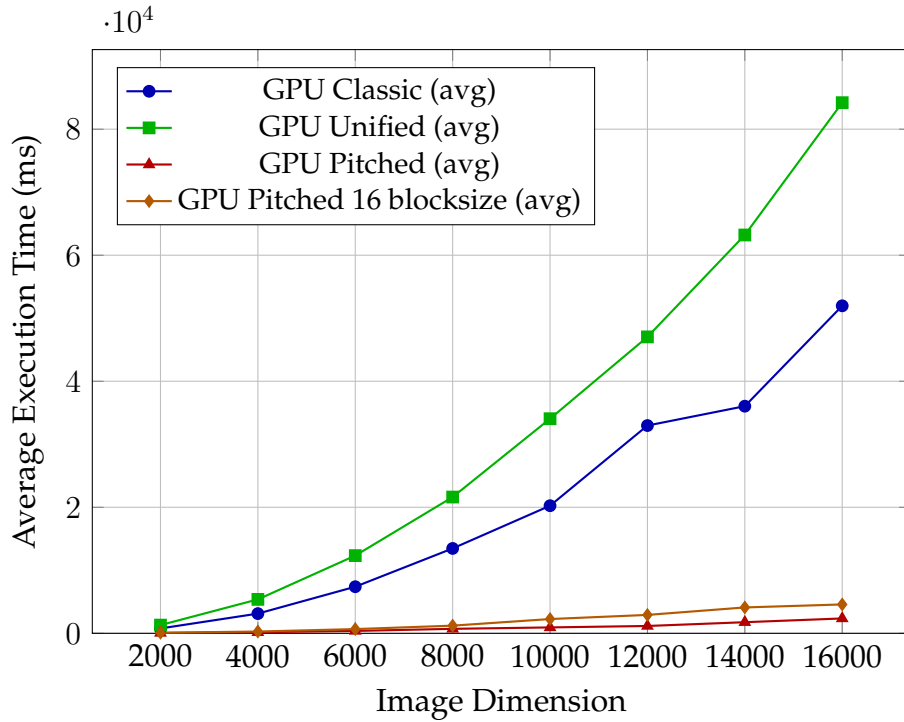


Figure 4.7: Average execution time comparison between all 4 GPU memory implementations

4.3.2 Interpretation of Results

The comparative analysis of the four GPU memory strategies—Classic Memory, Unified Memory, Pitched Memory, and Pitched Memory with a 16x16 block size—reveals clear trends in both performance and scalability.

Across all tests, Unified Memory consistently underperformed compared to Classic Memory, often by a factor of 1.5× to 2×. This is expected, given that Unified Memory simplifies memory management by abstracting away explicit host-device transfers, trading raw performance for development ease and stability. Despite the performance penalty, its reliability may justify its use in scenarios where manual memory management is error-prone or difficult.

The Pitched Memory implementation outperformed both Classic and Unified Memory approaches by a significant margin. Execution times were considerably lower across all dimensions and iteration counts. This confirms the efficiency of pitched memory for two-dimensional data alignment and access on the GPU.

When comparing Pitched Memory with and without 16×16 block size optimization, the results showed that the 16-block version was slightly slower in most scenarios. However, an interesting trend emerged at larger scales. During an additional testing session, at 20000×20000 resolution and high iteration counts, the 16-block variant began to outperform itself compared to previous sizes, suggesting that this layout may scale better with increasingly large workloads, potentially due to improved thread distribution and memory access patterns.

4.4 Discussion and Future Directions

4.4.1 Overview and Achievements

Throughout this project, we implemented and evaluated four different approaches for rendering fractals on the GPU: a classic memory model, a unified memory model, pitched memory, and a pitched memory model with smaller thread blocks. While all approaches achieved the goal of producing visually accurate Julia Set fractals, their performance varied significantly depending on the memory strategy and kernel configuration used.

The evolution of performance across versions was not strictly linear. Early versions focused on getting a working CUDA implementation, and the optimization direction was not fully defined. This resulted in performance fluctuations as new features, like different memory models or kernel parameters, were introduced without being tailored for a specific performance target. However, this experimental flexibility allowed a wide exploration of trade-offs between readability, stability, and

speed — eventually leading to significant improvements with the pitched memory approach.

4.4.2 Optimization Possibilities and Future Work

While the current implementations deliver strong performance, especially in the pitched memory version, there are still several optimization opportunities and areas for future development.

One potential improvement lies in optimizing kernel memory access patterns further, particularly through fine-tuned shared memory usage or better cache utilization. Kernel launches could also benefit from adaptive thread block sizing based on the image dimensions or dynamic parallelism techniques.

A key trade-off observed was between performance and code stability or portability. Unified memory, while slower, simplified memory management and could be valuable in more complex applications or educational contexts where ease of use matters more than raw speed. These types of trade-offs — between speed, maintainability, and extensibility — offer valuable lessons and decision points for future development.

Interactivity is another area with rich potential that was not touched yet. Future iterations of the application could include real-time fractal exploration, allowing users to zoom, pan, or change parameters like the complex constant or iteration depth on the fly. Implementing an interactive UI would require careful attention to performance to maintain responsiveness. Additionally, expanding color mapping options — such as gradient maps, user-defined palettes, or even procedural coloring based on iteration patterns — would significantly enhance both usability and visual appeal.

Overall, this project laid the groundwork for a modular and extensible fractal renderer. The exploration of multiple GPU memory strategies provided both a technical benchmark and a springboard for future experiments, and the flexible design encourages further enhancements in both performance and user experience.

Chapter 5

Conclusions

Fractals represent a fascinating intersection of mathematics, nature, and art, characterized by self-similarity and complex patterns emerging from simple rules. This thesis has aimed to deepen the understanding of fractals by exploring their mathematical foundations, various types, and wide-ranging applications. Beyond theory, fractals reveal profound insights into the natural world and offer powerful tools for modeling complex phenomena.

A major focus of this work has been the practical computation and visualization of fractals using GPU parallel processing. While the initial implementation of the Julia set fractal renderer was based on an existing CUDA example, this thesis expanded on it by developing a corresponding CPU version for direct performance comparison. Furthermore, several optimization strategies were explored and implemented to improve the efficiency of the GPU program, accompanied by detailed benchmarking to evaluate their impact.

Although the contributions may be incremental, they highlight the challenges and opportunities in fractal computation on modern hardware. The experimentation with GPU optimization techniques provides a valuable learning experience and sets the groundwork for future improvements in fractal rendering performance and quality. This work underscores the importance of combining mathematical insight with computational skills to effectively harness the potential of fractals in scientific and artistic contexts.

In summary, this thesis bridges theoretical knowledge and practical application, demonstrating how fractals can be both studied mathematically and realized efficiently through parallel computing. The journey through fractal theory, GPU programming, and optimization enriches the understanding of these complex structures and encourages continued exploration in this vibrant interdisciplinary field.

Bibliography

- [Add97] Paul S. Addison. *Fractals and Chaos*. Institute of Physics Publishing, 1997.
- [Fed88] Jens Feder. *Fractals*. Springer Science & Business Media, 1988.
- [GS10] Denny Gulick and Jon Scott, editors. *The Beauty of Fractals: Six Different Views*. MAA - The Mathematical Association of America (Incorporated), USA, 2010.
- [NA22] Cătălin Silviu Nuțu and Tiberiu Axinte. Fractals. origins and history. *Journal of Marine Technology & Environment*, 2:48, 2022.
- [Pac09] Luca Pacioli. *De Divina Proportione*. Venice, 1509.
- [PJS04] Heinz-Otto Peitgen, Hartmut Jürgens, and Dietmar Saupe. *Chaos and Fractals: New Frontiers of Science*. Springer, New York, 2nd edition, 2004. eBook ISBN: 0-387-21823-8.
- [SK10] Jason Sanders and Edward Kandrot, editors. *CUDA by Example: An Introduction to General-Purpose GPU Programming*. Addison-Wesley, USA, 2010.
- [VL15] Patricio Gonzalez Vivo and Jen Lowe. The book of shaders. <https://thebookofshaders.com/>, 2015. Accessed: 2025-06-15.